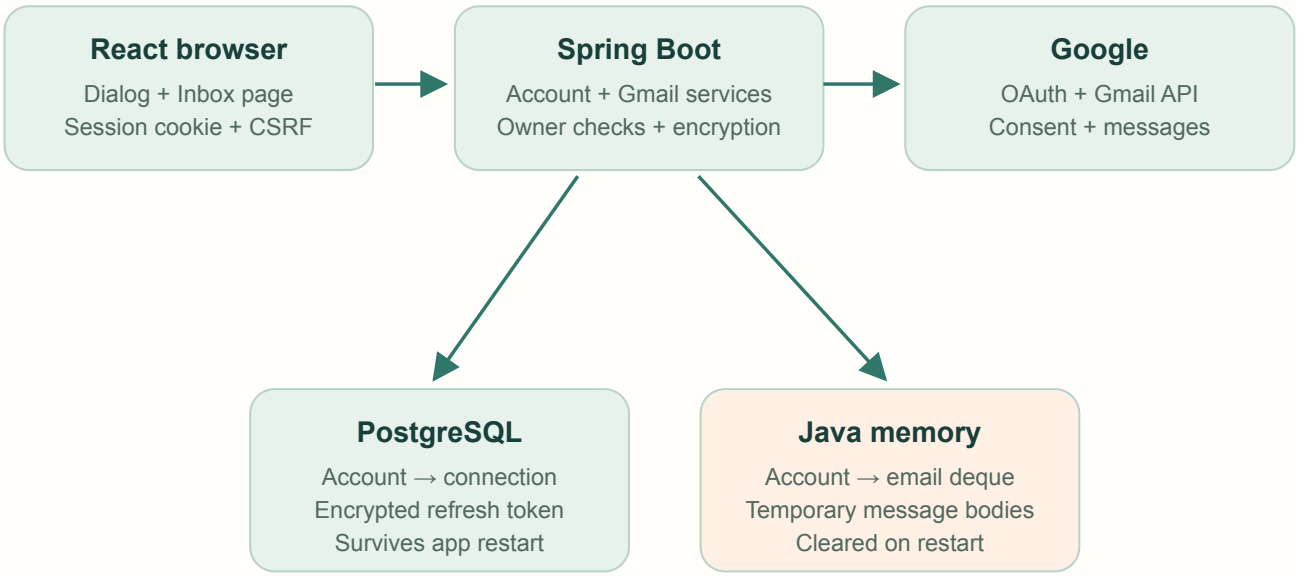
Gmail, from click to inbox.

A guided explanation of JobGraph's connection, message fetching, and security.



The AI service is not called by the current Fetch job alerts path.

OAuth flow | Real code | Security | Debugging

Prepared for your current JobGraph codebase • 03 October 2026

How to use this guide

Read the diagrams first, then follow the numbered source excerpts in your IDE. Examples use fictional addresses and placeholders. No real keys, passwords, tokens, or mailbox contents are included.

Your reading route

Follow the user's journey first. Return to the reference pages while debugging.

Pages	What you will understand
3–4	The architecture, account ownership, and the names of every credential.
5–10	The connection dialog, sessions, OAuth state, Google consent, and callback validation.
11–14	Exchanging the code, confirming the Gmail address, saving tokens, encryption, and automatic refresh.
15–20	Connection lifecycle, Gmail search, message bodies, temporary storage, and the Inbox UI.
21–25	Security boundaries, endpoint reference, local setup, and diagnosing errors.
26–30	Current limitations, a debugger walkthrough, a worked example, tests, and source references.

The feature that exists today

A signed-in user saves a profile, connects one Gmail account through Google, and manually fetches messages matching a job-alert search. The Inbox page displays those fetched messages. There is no automatic profile relevance ranking in this request path.

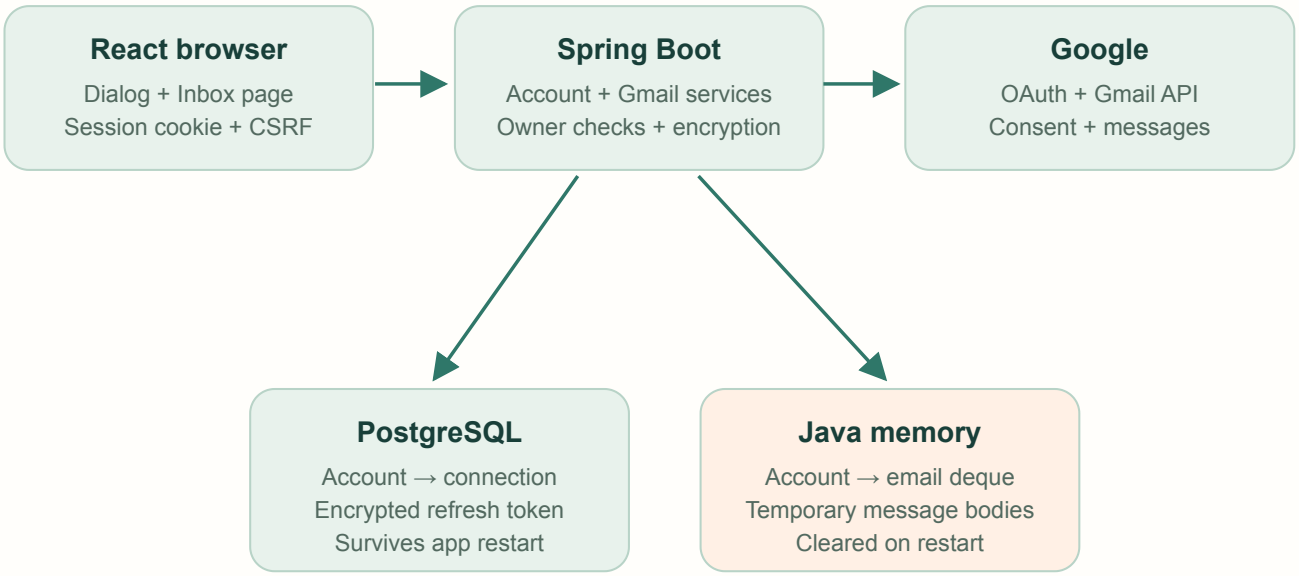
Three actions to keep separate

1. **Connect:** obtain permission and save a reusable credential. This does not fetch the job-alert list.
2. **Fetch:** use that credential to search Gmail and load message details into Java memory.
3. **View:** read the already-fetched messages from JobGraph's API and render them in React.

> **About the excerpts**
> Code blocks with line numbers are copied from the working tree used to build this guide. Blocks marked 'illustrative example' explain a concept and are not additional application code. The source manifest records the files and their hashes.

Who does what?

The browser starts the flow; the backend owns the Google credentials and mailbox requests.



The AI service is not called by the current Fetch job alerts path.

Component	Responsibility
React + Vite	Collect the intended Gmail address, send authenticated API requests, redirect the browser to Google, and display messages.
Spring Security	Restore the user's session, check CSRF tokens on writes, and enforce protected API routes.
GmailConnectionController	Bind the OAuth attempt to a session/account; validate the callback; manage connection status and disconnect.
GmailAuthService + TokenCipher	Create Google clients, exchange credentials, verify the mailbox address, and encrypt/decrypt the saved refresh token.
GmailFetchService + EmailStack	Search and read Gmail messages; keep separate temporary collections for each account.

Where the AI service fits

The Python service has a /filter-emails endpoint, but EmailController and the current frontend fetch flow do not call it. The ranking feature is therefore separate from the flow documented here.

The names that matter

Each credential has a specific owner, purpose, and place to live.

Name	Meaning and current storage
JobGraph login email	Identifies the local account. Its password is BCrypt-hashed in the accounts table.
Connected Gmail address	The mailbox Google confirms. May differ from both the login email and the resume's contact email.
Session cookie	An identifier that lets the backend recover the JobGraph login session. It is not a Gmail access token.
Spring CSRF token	A session-backed token the frontend sends with writes such as POST /gmail/connect.
OAuth state	Random value saved before leaving for Google. The callback must return the same value in the same session.
Authorization code	A temporary value returned through the browser. The backend exchanges it for Google tokens.
Access token	Credential used on Gmail API requests. The Java client holds it in memory.
Refresh token	Credential used to obtain a new access token. JobGraph saves it encrypted in PostgreSQL.
OAuth client ID / secret	Identify the JobGraph web application to Google. The secret is backend configuration.
Token encryption key	JobGraph's own 32-byte AES key. It protects stored refresh tokens and is independent of the Google client secret.

Account ownership

A profile is required before connecting, but the Gmail connection belongs to the Account entity. Editing the resume contact email does not change the Gmail connection or its owner.

1. Start in the browser

Connect Gmail opens a dialog. Continue with Google starts the server flow.

GmailConnectDialog uses a native dialog and focuses the email input. The form trims the address and calls onContinue. HTML validates email syntax; the backend also validates it. These checks do not establish ownership of the mailbox.

ConnectGmailButton.jsx — lines 36–42 · exact excerpt

```
36 async function connect(email) {
37   setBusy('connecting'); setDialogError(''); setNotice('')
38   try {
39     // Google handles sign-in and any verification code before granting Gmail access.
40     const { url } = await startGmailConnection(email)
41     window.location.assign(url)
42   } catch (requestError) { setDialogError(requestError.message || 'Could not start Gmail
   connection.'); setBusy('') }
```

startGmailConnection sends JSON to `POST /api/gmail/connect`. The response contains an authorization URL. `window.location.assign(url)` navigates the browser to Google's website; it is not an AJAX request to Google's login screen.

Request and response shape — illustrative example

```
POST /api/gmail/connect
Content-Type: application/json
X-CSRF-TOKEN: <session CSRF token>
Cookie: JSESSIONID=<local session id>

{"email":"career.mailbox@gmail.com"}

200 OK
{"url":"https://accounts.google.com/o/oauth2/auth?..."}
```

Why an email field is not enough

Typing an address only tells JobGraph which mailbox the user intends to connect. Google handles sign-in, any verification challenge, and the permission screen. JobGraph does not send its own Gmail verification code and never asks the user to enter a Google password into this dialog.

> **After returning from Google**
>
> The URL result such as ?gmail=connected controls a UI notice. The component also calls GET /api/gmail/status for the saved connection. A success-looking URL alone is not proof that a connection exists.

2. Keep the user's identity

Session authentication protects local API calls; CSRF protects state-changing requests.

api.js lines 29–38 · exact excerpt

```
async function request(path, options = {}) {
  const method = options.method || 'GET'
  const headers = new Headers(options.headers || {})
  // Read requests need no CSRF header; writes do.
  if (!['GET', 'HEAD', 'OPTIONS'].includes(method.toUpperCase())) {
    const token = await csrfToken()
    headers.set(token.headerName, token.token)
  }
  const response = await fetch(`${API_BASE}${path}`, { ...options, method, headers, credentials:
'same-origin' })
  if (!response.ok) throw await errorFromResponse(response, 'Request failed')
```

Before a write, csrfToken() calls GET /api/auth/csrf if needed. It receives a token and the header name expected by Spring. The browser sends the session cookie with same-origin requests. Vite proxies /api to localhost:8080, so React normally uses relative URLs.

AuthController.java lines 90–98 · exact excerpt

```
Authentication authenticated = manager.authenticate(new UsernamePasswordAuthenticationToken(email,
password));
// Give the authenticated user a new session ID to prevent session reuse.
if (request.getSession(false) != null) request.changeSessionId();
SecurityContext context = SecurityContextHolder.createEmptyContext();
context.setAuthentication(authenticated);
SecurityContextHolder.setContext(context);
contexts.saveContext(context, request, response);
// Login changes the session, so the frontend must request a fresh CSRF token.
csrfTokens.saveToken(null, request, response);
```

Login verifies credentials, rotates an existing session ID, puts the authenticated user in a SecurityContext, and explicitly saves it. It clears the previous CSRF token; the frontend clears its cached token too. A subsequent write obtains a fresh token.

Check	Question it answers
Session authentication	Which JobGraph account is making this request?
Spring CSRF token	Does this write include the token expected for this session?
OAuth state, later	Does Google's callback match the connection attempt started in this session?

GET requests do not need Spring's CSRF header here. The OAuth callback is a GET, so it uses the separate state validation shown on page 10. Public signup/login POST routes still require CSRF. [S1]

3. Create the OAuth attempt

The backend chooses the account from the authenticated session.

GmailConnectionController.java lines 59–71 · exact excerpt

```
59 Account account = accounts.current(authentication);
60 if (!profiles.exists(account.getId())) throw new ResponseStatusException(HttpStatus.CONFLICT, "Save a
   profile first");
61 String requestedEmail = body.email().trim();
62 byte[] nonce = new byte[32];
63 random.nextBytes(nonce);
64 String state = Base64.getUrlEncoder().withoutPadding().encodeToString(nonce);
65 try {
66     String url = gmail.authorizationUrl(state, requestedEmail);
67     // Remember who started OAuth so the callback cannot attach Gmail to another account.
68     request.getSession().setAttribute("gmailOAuthState", state);
69     request.getSession().setAttribute("gmailOAuthAccountId", account.getId());
70     request.getSession().setAttribute("gmailOAuthEmail", requestedEmail);
71     return Map.of("url", url);
```

1. **Find the owner:** accounts.current(authentication) resolves the logged-in account. The browser does not supply an accountId.
2. **Require a saved profile:** the controller returns 409 if the profile does not exist.
3. **Create state:** SecureRandom fills 32 bytes. URL-safe Base64 makes those bytes suitable for a redirect URL. Base64 is an encoding step, not encryption.
4. **Build Google's URL:** authorizationUrl() also checks that the encryption key is configured before redirecting the user.
5. **Save three session values:** state, account ID, and requested email. These tie the future callback to this exact attempt.

> **Why these three values?**
> State connects the outbound and return trips. Account ID checks that the callback is still for the same JobGraph user. Requested email lets the backend compare Google's confirmed mailbox with the address the user entered.

The values are one set of session attributes, so starting another connection in a second tab overwrites the first attempt. There is no separate OAuth-attempt timestamp or short expiry in this code; the attempt depends on session lifetime and removal during callback.

4. Build Google's consent URL

The Java client builds the URL from the app credentials and requested permission.

GmailAuthService.java lines 58–68 · exact excerpt

```
58      return new GoogleAuthorizationCodeFlow.Builder(
59              GoogleNetHttpTransport.newTrustedTransport(), GsonFactory.getDefaultInstance(),
60              secrets, List.of(GmailScopes.GMAIL_READONLY))
61              .setAccessType("offline").build();
62  }
63
64  public String authorizationUrl(String state, String email) throws IOException,
    GeneralSecurityException {
65      cipher.verifyConfigured();
66      // Google may prefill this address, but the user still approves access there.
67      return flow().newAuthorizationUrl().setRedirectUri(redirectUri).setState(state)
68              .set("login_hint", email).set("prompt", "consent select_account").build();
```

Parameter / setting	Why the code uses it
client_id	Identifies the configured Google web OAuth client.
redirect_uri	The backend callback address registered for that client.
response_type=code	The authorization-code flow returns a code to the callback.
gmail.readonly	Requests permission to read Gmail. This scope does not grant sending, deletion, or marking messages read.
access_type=offline	Requests the ability to obtain fresh access tokens later through a refresh token.
state	Carries the random value created by the backend.
login_hint	Suggests the typed address to Google; it is not a mailbox ownership check.
prompt=consent select_account	Requests consent and an account picker.

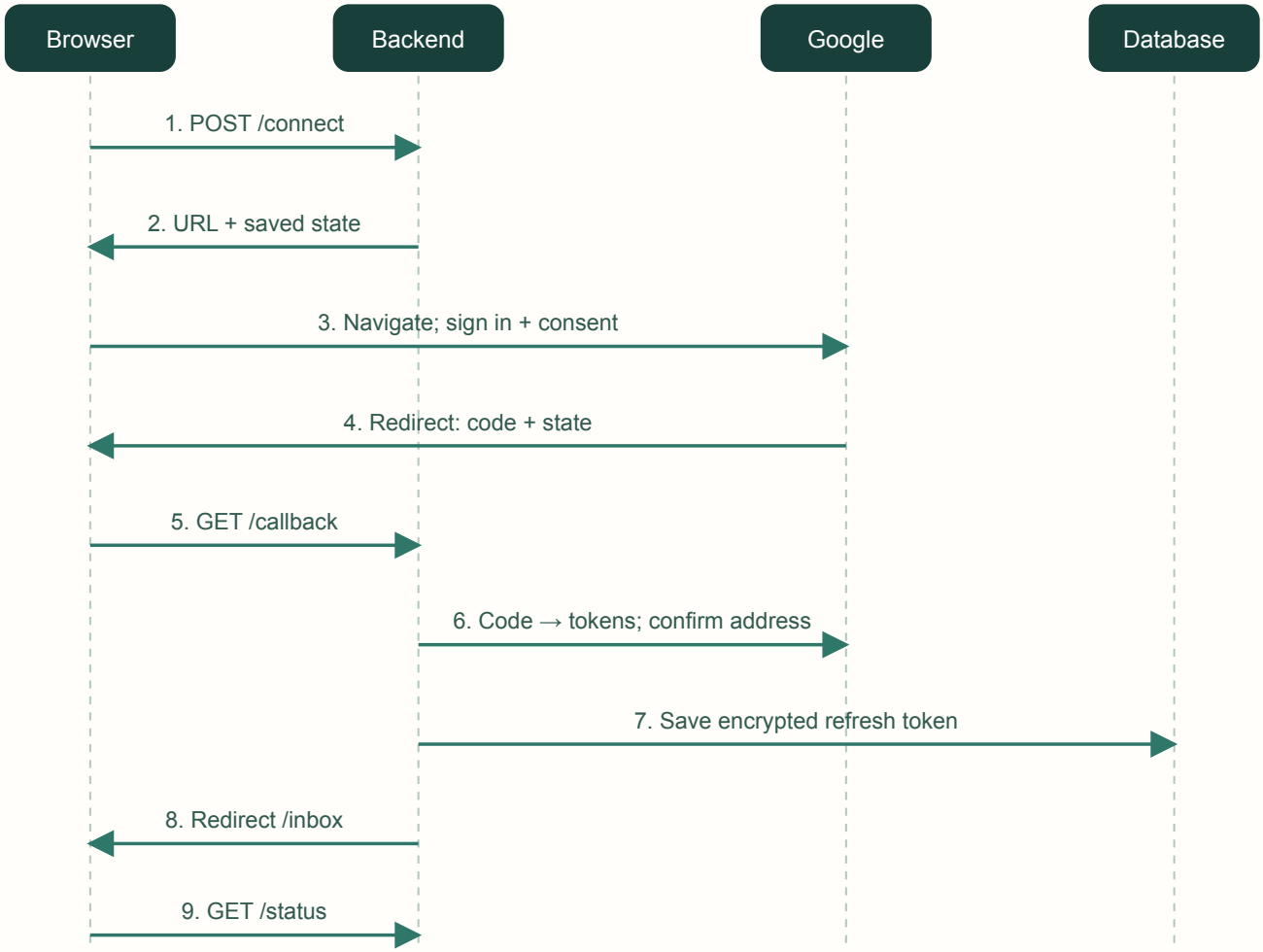
The installed google-api-client 2.7.0 builds an authorization URL beginning with `https://accounts.google.com/o/oauth2/auth`. The token exchange uses `https://oauth2.googleapis.com/token`. These endpoints were checked in the installed library's GoogleOAuthConstants source.

> **Read access is broader than the search**
>
> gmail.readonly permits reading mailbox data. The job-alert query controls what this feature fetches; it does not limit the underlying OAuth grant to job emails.

Repo: GmailAuthService:48–68. Protocol background [G1] [G2]

The complete connection trip

Follow the arrows once before reading the callback code.



Google returns the browser to the backend callback with a code and state. The browser's JobGraph cookie allows the backend to recover the session that started the flow. The backend then talks directly to Google to exchange the code and identify the Gmail mailbox.

What the browser receives

It receives the authorization URL, the callback code/state while navigating, a redirect to /inbox, and connection status. JobGraph's JSON APIs do not return the Google access token, refresh token, client secret, or encryption key.

5. Validate the callback

The callback can reach the controller anonymously, but attaching Gmail still needs the original login session.

SecurityConfig permits GET /api/gmail/callback so the controller can handle success, cancellation, and expired sessions. The controller immediately checks for a session and authentication; without them it redirects to /login?gmail=session-expired.

GmailConnectionController.java lines 89–103 · exact excerpt

```
89  Object expected = session.getAttribute("gmailOAuthState");
90  Object startedFor = session.getAttribute("gmailOAuthAccountId");
91  Object requestedEmail = session.getAttribute("gmailOAuthEmail");
92  // A state value can only be used once.
93  session.removeAttribute("gmailOAuthState");
94  session.removeAttribute("gmailOAuthAccountId");
95  session.removeAttribute("gmailOAuthEmail");
96  Account account = accounts.current(authentication);
97  // Reject callbacks from another browser session or account.
98  if (!(expected instanceof String expectedState) || state == null ||
99          !MessageDigest.isEqual(expectedState.getBytes(java.nio.charset.StandardCharsets.UTF_8),
100                 state.getBytes(java.nio.charset.StandardCharsets.UTF_8)) ||
101         !account.getId().equals(startedFor) || !(requestedEmail instanceof String)) {
102     response.sendRedirect(frontendUrl + "/inbox?gmail=invalid-state");
103     return;
```

The callback reads all three saved values and removes them before proceeding. It compares the saved and returned state using MessageDigest.isEqual, checks the account ID, and requires the requested email. A mismatch returns /inbox?gmail=invalid-state without exchanging the code.

Then choose the outcome

- If Google returned an error, or no code, redirect with `gmail=cancelled`.
- If checks pass, call `gmail.complete(account, code, requestedEmail)`.
- After completion, clear that account's old email stack and redirect with `gmail=connected`.
- If the confirmed email differs, use `gmail=email-mismatch`. Other completion failures use `gmail=error`.

> **An attempt is consumed on return**
>
> Even a bad-state callback removes the saved attempt. Retrying the same callback URL will not complete that attempt. Begin again with Connect Gmail.

6. Exchange the code

The backend obtains tokens, then asks Gmail which mailbox those tokens belong to.

GmailAuthService.java lines 73–88 · exact excerpt

```
73 public String complete(Account account, String code, String requestedEmail) throws IOException,
   GeneralSecurityException {
74     TokenResponse token = flow().newTokenRequest(code).setRedirectUri(redirectUri).execute();
75     Gmail gmail = gmailWithTokens(token.getAccessToken(), token.getRefreshToken());
76     String address = gmail.users().getProfile("me").execute().getEmailAddress();
77     if (address == null || address.isBlank()) throw new IOException("Google did not return a Gmail
   address");
78     if (!address.equalsIgnoreCase(requestedEmail)) throw new GmailAddressMismatchException();
79
80     Optional<GmailConnection> previous = connections.findByAccount_Id(account.getId());
81     String refreshToken = token.getRefreshToken();
82     // Google may omit a new refresh token when reconnecting the same inbox.
83     if ((refreshToken == null || refreshToken.isBlank()) && previous.isPresent()
84             && previous.get().getGmailAddress().equalsIgnoreCase(address)) {
85         refreshToken = cipher.decrypt(previous.get().getEncryptedRefreshToken());
86     }
87     if (refreshToken == null || refreshToken.isBlank()) {
88         throw new IOException("Google did not provide offline access. Disconnect in Google and retry
   consent.");
```

1. **Exchange:** newTokenRequest(code).execute() contacts Google’s token endpoint using the configured client and callback URI.
2. **Create a client:** gmailWithTokens receives the returned access and refresh tokens.
3. **Confirm identity:** users().getProfile("me") reads the mailbox’s email address. Here ‘me’ means the user authorized by the Google credential.
4. **Compare:** the confirmed address must equal the typed address, ignoring case. A login hint did not enforce this earlier.
5. **Ensure durable access:** keep the new refresh token, or reuse the existing decrypted token only when reconnecting the same Gmail address.

> **Why reuse only for the same address?**
>
> A refresh token belongs to a particular authorization. Reusing mailbox A’s token while displaying mailbox B’s address would attach the wrong mailbox. A different mailbox without a new refresh token causes completion to fail.

The address comparison does not normalize Gmail dots, plus aliases, or every possible Workspace alias. A typed alias that differs from Google’s confirmed address can produce an email-mismatch result. Use the address Google confirms.

7. Save the connection

One database row connects a JobGraph account to one Gmail mailbox.

GmailAuthService.java lines 90–97 · exact excerpt

```
90 GmailConnection connection = previous.orElseGet(GmailConnection::new);
91 connection.setAccount(account);
92 connection.setGmailAddress(address);
93 // Only the encrypted refresh token is stored; short-lived access tokens stay in memory.
94 connection.setEncryptedRefreshToken(cipher.encrypt(refreshToken));
95 connection.setUpdatedAt(LocalDateTime.now());
96 connections.save(connection);
97 return address;
```

GmailConnection.java lines 12–14 · exact excerpt

```
12 @OneToOne(fetch = FetchType.LAZY, optional = false)
13 @JoinColumn(name = "account_id", nullable = false, unique = true)
14 private Account account;
```

The service updates the existing row or creates a new GmailConnection. The database requires a unique, non-null account_id. That means an account has at most one saved Gmail connection; the saved profile and Gmail row both point to the account.

Database column	Stored value
account_id	The JobGraph account owner; it is the lookup key used by the services.
gmail_address	The address returned by Google's Gmail profile API.
encrypted_refresh_token	Base64 text containing the IV, encrypted refresh token, and authentication tag.
updated_at	When JobGraph last saved/replaced the connection; this is not a token-expiry time.

GmailConnectionRepository extends JpaRepository, so save() persists the entity. `findByAccount_Id(accountId)` means lookup by the nested `account.id` property and returns Optional<GmailConnection>. The underscore makes the property traversal explicit.

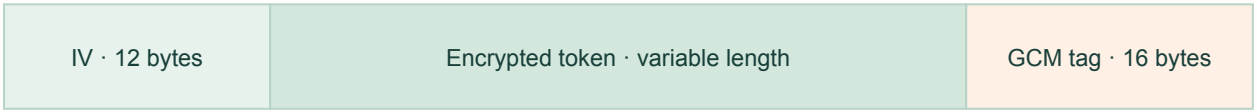
> **A detail in the schema**
>
> gmail_address itself is not unique. The schema allows the same Gmail address to be connected to separate JobGraph accounts after each completes consent. Ownership checks still use account_id.

8. Protect the saved token

TokenCipher uses AES-256-GCM so the backend can recover the refresh token when needed.



Database value after Base64 encoding



The encryption key stays in backend configuration, outside this row.

TokenCipher.java — lines 34–45 · exact excerpt

```
34 public String encrypt(String plaintext) {
35     try {
36         // A fresh IV makes repeated encryptions of the same token different.
37         byte[] iv = new byte[12];
38         random.nextBytes(iv);
39         Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
40         cipher.init(Cipher.ENCRYPT_MODE, key(), new GCMParameterSpec(128, iv));
41         byte[] encrypted = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));
42         byte[] combined = new byte[iv.length + encrypted.length];
43         System.arraycopy(iv, 0, combined, 0, iv.length);
44         System.arraycopy(encrypted, 0, combined, iv.length, encrypted.length);
45         return Base64.getEncoder().encodeToString(combined);
```

The configured key is Base64-decoded and must be exactly 32 bytes: a 256-bit AES key. Every encryption generates a fresh 12-byte IV. GCMParameterSpec(128, iv) requests a 128-bit authentication tag. Java's doFinal() produces encrypted bytes with the tag; the code prepends the IV and encodes the combined bytes as Base64.

During decryption, the first 12 bytes become the IV and the remaining bytes go through AES/GCM with the same key. A wrong key or changed encrypted value should fail authentication instead of producing a trusted token.

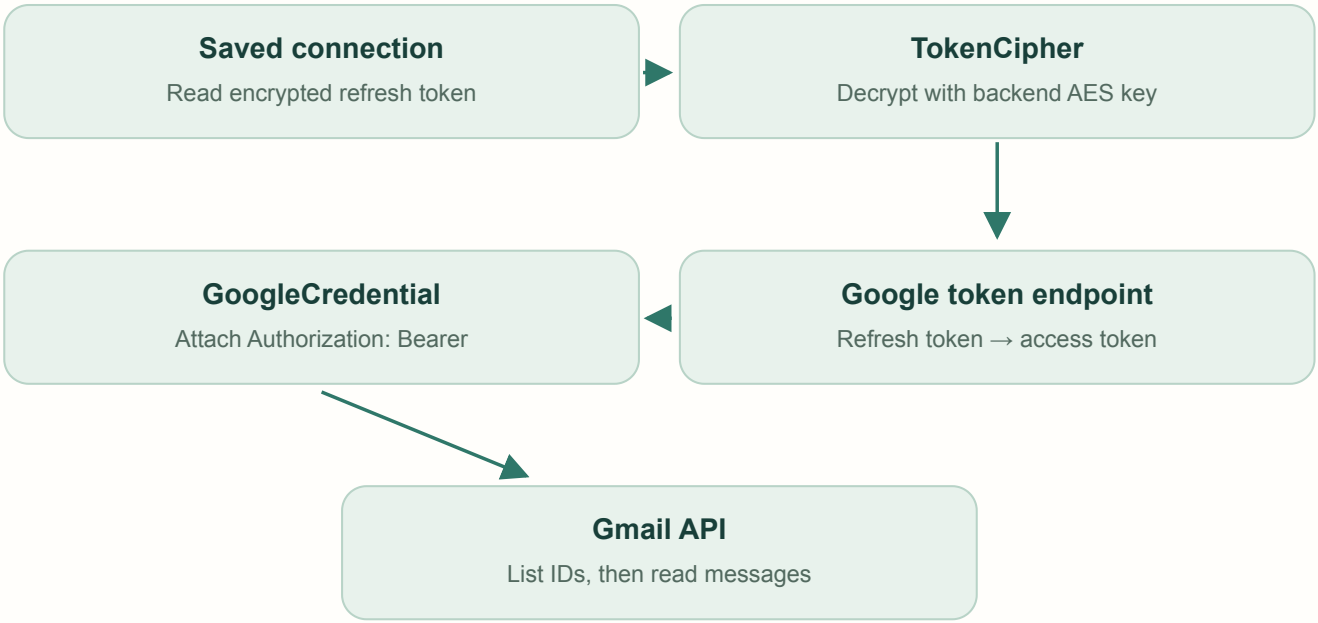
> **Encryption and hashing serve different needs**
>
> JobGraph must recover the refresh token to send it to Google, so it encrypts it. A login password is only checked for a match, so it is stored as a BCrypt hash. Base64 alone provides neither protection.

9. Obtain access when fetching

The saved refresh token lets the Google client obtain a usable access token.

GmailAuthService.java lines 111–116 · exact excerpt

```
111 public Gmail getGmailService(Integer accountId) throws IOException, GeneralSecurityException {
112     // Build the Gmail client from this account's saved connection.
113     GmailConnection connection = connections.findByAccount_Id(accountId)
114             .orElseThrow(() -> new IllegalStateException("Connect Gmail before fetching emails"));
115     String refreshToken = cipher.decrypt(connection.getEncryptedRefreshToken());
116     return gmailWithTokens(null, refreshToken);
```



gmailWithTokens(null, refreshToken) constructs GoogleCredential with the app's client ID/secret and the decrypted refresh token. It intentionally starts with no access token. On the first outgoing request, the library's Credential interceptor sees the missing access token and refreshes it, then adds the Bearer authorization header.

This behavior was checked in the installed google-oauth-client 1.36.0 source, Credential.intercept(), lines 195–207. The library can also refresh around expiry or authentication failure; the application does not implement its own refresh HTTP request. [J1]

> **Current caching behavior**
>
> getGmailService() creates a new client on each fetch call. The access token is reused within that client's list/get operations, but there is no shared per-account access-token cache across fetch calls. The refresh token remains encrypted in the database.

Status, replacement, disconnect

A saved connection, an active login, and a valid Google grant are separate states.

GmailAuthService.java lines 119–121 · exact excerpt

```
119 public Optional<String> connectedAddress(Integer accountId) {
120     return connections.findByAccount_Id(accountId).map(GmailConnection::getGmailAddress);
121 }
```

connectedAddress() checks the local database and maps the connection to its address. It does not decrypt a token or contact Google. Therefore ‘Connected’ means a row exists; it does not prove the Google grant is still valid.

Action	What happens in the current code
Check status	GET /gmail/status returns connected=true plus the stored address, or false and null.
Change Gmail account	Run the same consent flow. The row is updated after address/token checks succeed; old fetched messages are then cleared.
Cancel replacement	No new connection is saved. The old saved connection remains.
Disconnect	DELETE /gmail/connection deletes the account’s local connection row and clears its in-memory email stack.
Log out	Invalidate the JobGraph session. The Gmail connection row remains for the next login.
Restart backend	Saved connection rows remain in PostgreSQL. EmailStack’s in-memory messages disappear.

GmailConnectionController.java lines 124–128 · exact excerpt

```
124 public ResponseEntity<Void> disconnect(Authentication authentication) {
125     Integer accountId = accounts.current(authentication).getId();
126     gmail.disconnect(accountId);
127     stacks.clear(accountId);
128     return ResponseEntity.noContent().build();
```

Local disconnect is not Google revocation

There is no call to Google’s revocation endpoint. Disconnect removes JobGraph’s saved token; the authorization grant may still appear in the user’s Google Account settings. An already-running request may also finish after disconnect.

10. Click Fetch job alerts

The backend searches the signed-in user's connected Gmail account.

EmailController.java — lines 34–46 · exact excerpt

```
34      @RequestParam(defaultValue = "10") int maxResults,
35      Authentication authentication) {
36  Integer accountId = accounts.current(authentication).getId();
37  if (gmail.connectedAddress(accountId).isEmpty())
38      throw new ResponseStatusException(HttpStatus.CONFLICT, "Connect Gmail first");
39  try {
40      List<EmailDto> emails = fetcher.fetchEmails(accountId, query, Math.max(1, Math.min(maxResults,
    50)));
41      // Keep fetched messages under this account's own stack.
42      stack.pushAll(accountId, emails);
43      return ResponseEntity.ok(java.util.Map.of("fetched", emails.size(), "stackSize",
    stack.size(accountId)));
44  } catch (Exception e) {
45      log.error("Gmail fetch failed for account {}", accountId, e);
46      return ResponseEntity.status(HttpStatus.BAD_GATEWAY).body("Could not fetch Gmail messages.
    Reconnect Gmail if access was revoked.");
```

The frontend uses POST /api/emails/fetch with its session and CSRF header. AccountService derives accountId from authentication. The controller first checks that a local Gmail connection exists, then calls GmailFetchService. Results are pushed into the account's stack, and the response reports counts.

Default query, line breaks added for readability — illustrative example

```
newer_than:10d (
  subject:(job OR opening OR hiring OR position
           OR vacancy OR recruitment OR interview)
  OR from:(glassdoor.com OR indeed.com
           OR linkedin.com OR naukri.com)
)
```

The default search asks for recent messages with job-related subject words or matching senders. maxResults defaults to 10, and the controller clamps it to 1–50. The current button sends no custom query or limit. These are application choices, not Gmail's maximum capabilities.

> **Fetching does not guarantee a relevant job**
>
> These search terms can include marketing, interview notifications, or unrelated messages. A From header is not proof of sender authenticity. This path does not perform profile matching, fraud checks, or AI scoring.

11. Search IDs, then read details

The Gmail API list call returns references; each get call supplies a message.

GmailFetchService.java lines 31–46 · exact excerpt

```
31 public List<EmailDto> fetchEmails(Integer accountId, String query, int maxResults) throws Exception {
32     // The account ID selects that user's saved Gmail connection and token.
33     Gmail service = gmailAuthService.getGmailService(accountId);
34     List<EmailDto> results = new ArrayList<>();
35
36     var response = service.users().messages().list("me")
37             .setQ(query)
38             .setMaxResults((long) maxResults)
39             .execute();
40
41     var messages = response.getMessages();
42     if (messages == null) return results;
43
44     for (var msgRef : messages) {
45         Message message = service.users().messages().get("me", msgRef.getId()).execute();
46         results.add(toEmailDto(message));
```

Java call	Conceptual REST request
messages().list("me")	GET https://gmail.googleapis.com/gmail/v1/users/me/messages?q=…&maxResults=…
messages().get("me", id)	GET https://gmail.googleapis.com/gmail/v1/users/me/messages/{id}
users().getProfile("me")	GET https://gmail.googleapis.com/gmail/v1/users/me/profile (used during connection)

The list response provides IDs and thread IDs, so the loop requests each message separately. ‘me’ always refers to the Google user behind that client’s credential. Ten returned messages require one list call and ten detail calls, plus a token refresh when needed. [G3] [G4]

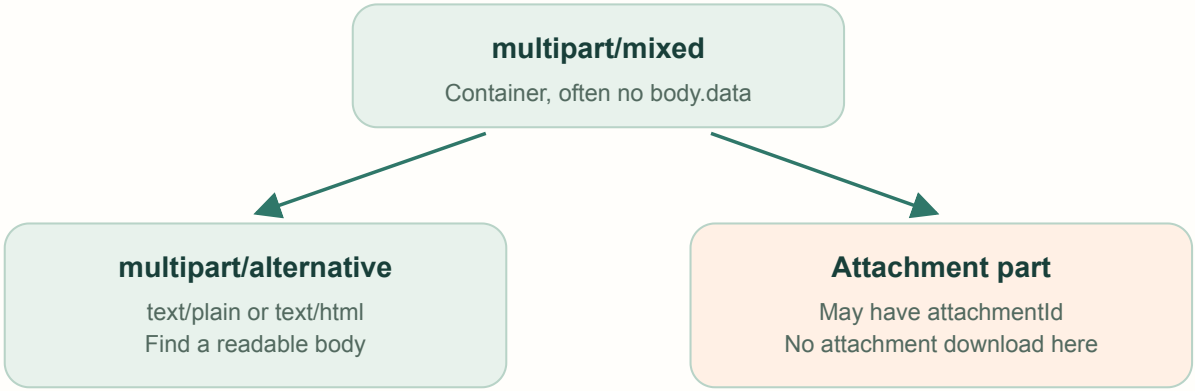
What the loop does and does not do

- If getMessages() returns null, return an empty list.
- Read detail requests sequentially, then convert each result to EmailDto.
- Sort the fetched batch oldest to newest before passing it to the stack.
- There is no nextPageToken loop, so this fetch reads one result page.
- There is no partial-success handling: if one message fails, the method throws and the controller does not push this batch.

The code does not send a format parameter to messages.get. The API’s parsed message response supplies payload and headers used by the converter. The code never requests Gmail mutations such as marking a message read.

12. Turn MIME into readable text

An email can contain nested parts, alternative formats, and attachments.



GmailFetchService.java lines 78–98 · exact excerpt

```
78 private String extractBody(MessagePart payload) {
79     if (payload == null) return "";
80
81     // Simple case: body directly on this part
82     if (payload.getBody() != null && payload.getBody().getData() != null) {
83         return decodeBase64(payload.getBody().getData());
84     }
85
86     // Multipart: look for a text/plain part
87     if (payload.getParts() != null) {
88         for (MessagePart part : payload.getParts()) {
89             if ("text/plain".equalsIgnoreCase(part.getMimeType())
90                     && part.getBody() != null && part.getBody().getData() != null) {
91                 return decodeBase64(part.getBody().getData());
92             }
93         }
94         // fallback: recurse into nested parts
95         for (MessagePart part : payload.getParts()) {
96             String nested = extractBody(part);
97             if (!nested.isEmpty()) return nested;
98         }
```

extractBody first decodes data on the current part. Otherwise it looks for a direct text/plain child. If it finds none, it recursively checks children and returns the first non-empty body. The data is URL-safe Base64 and is decoded as UTF-8.

> **Practical limits of this parser**
>
> It does not download attachmentId bodies, combine every part, or inspect character-set declarations. The fallback can return HTML text or another part's content. It prefers plain text among immediate children, but it is not a complete MIME normalization engine.

13. Keep each account's messages

EmailDto is the API shape; EmailStack is temporary server memory.

GmailFetchService.java lines 70–75 · exact excerpt

```
70 String body = extractBody(payload);
71 LocalDateTime receivedAt = Instant.ofEpochMilli(message.getInternalDate())
72         .atZone(ZoneId.systemDefault())
73         .toLocalDateTime();
74
75 return new EmailDto(message.getId(), subject, from, message.getSnippet(), body, receivedAt);
```

The converter reads Subject and From headers, takes Gmail's snippet and ID, extracts the body, and converts internalDate from milliseconds to LocalDateTime using the backend's time zone. Jackson serializes the DTO getters into the JSON the frontend receives.

EmailStack.java lines 12–24 · exact excerpt

```
12 // Each account has its own temporary inbox; entries disappear on restart.
13 private final ConcurrentHashMap<Integer, Deque<EmailDto>> stacks = new ConcurrentHashMap<>();
14
15 public void pushAll(Integer accountId, List<EmailDto> emails) {
16     Deque<EmailDto> stack = stacks.computeIfAbsent(accountId, ignored -> new ArrayDeque<>());
17     // ArrayDeque needs synchronization when requests for one account overlap.
18     synchronized (stack) {
19         for (EmailDto email : emails) {
20             // Fetching again refreshes a message instead of showing duplicates.
21             stack.removeIf(previous -> Objects.equals(previous.getMessageId(), email.getMessageId()));
22             stack.push(email);
23         }
24     }
```

ConcurrentHashMap separates collections by account ID. computeIfAbsent creates the deque only when needed. ArrayDeque itself is not thread-safe, so operations on an account's deque are synchronized. Before pushing a message, the code removes any existing item with the same Gmail message ID.

Order in a single fresh batch illustrative example

```
Fetched and sorted: [older A, newer B]
push(A) -> [A]
push(B) -> [B, A]    // newest is now at the front
```

> **Two limits to understand**
>
> Only the incoming batch is sorted. A later custom search that fetches old messages can move those older messages above newer cached ones. Also, the cache has no size cap, expiry, or database persistence.

14. Show the messages in React

Fetching and displaying use two API requests.

ConnectGmailButton.jsx lines 55–57 · exact excerpt

```
55 const result = await fetchEmails()
56 // The fetch endpoint returns a count; the stack endpoint holds the messages.
57 await onFetched?.()
```

The POST fetch response is a count object, not the message list. After it succeeds, onFetched() calls the handler supplied by InboxPage. That handler requests GET /api/emails/stack, puts the returned array into React state, and scrolls toward the message list.

InboxPage.jsx lines 37–46 · exact excerpt

```
37 async function showFetchedEmails() {
38   messagesChanged.current = true
39   setLoadingEmails(true)
40   setEmailError('')
41   try {
42     const items = await getEmailStack()
43     setEmails(Array.isArray(items) ? items : [])
44   } catch (error) {
45     setEmailError(error.message || 'Could not load your messages.')
46     throw error
```

How the Inbox page behaves

- On opening, load the saved profile and the existing stack. A user without a saved profile is redirected to the workspace.
- While loading, display a loading message; on failure, display the API error.
- For each email, show subject, sender, received time, snippet, and an expandable body.
- Disconnect clears the local React array after the backend removes the connection and stack.

Rendering pattern used by FetchedEmails illustrative example

```
<div className="fetched-email__body">{email.body}</div>
```

> **Email text stays text**
> The body is inserted as a React text value. The component does not use dangerouslySetInnerHTML. HTML-looking email text is therefore escaped rather than executed or rendered as remote HTML. This also means some HTML-only messages may look unformatted.

What protects which boundary?

Security is several checks working together, each at a different point in the request.

Boundary / risk	Protection in this code
Anonymous access to inbox APIs	SecurityConfig requires authentication for /api/** except its explicit public routes.
Forged browser write requests	Session-backed Spring CSRF token on connect, fetch, disconnect, login, signup, and logout.
Callback attached to the wrong attempt	Random OAuth state, saved account ID, requested email, and session checks.
Typed address falsely claimed as owned	Read Google's /users/me/profile result and compare the returned address.
Access to another JobGraph account's data	Derive the account from Authentication; repository and stack calls use that account ID.
Database-only theft of a refresh token	AES-GCM encryption with the key outside the database row.
Token exposure through frontend JSON	Return status/counts/messages, not the connection entity or token values.
Modifying Gmail accidentally	Request gmail.readonly and call read methods only.
Email content executing as page HTML	Render message body through ordinary React text interpolation.

Cookie and origin settings

The current properties set HttpOnly=true, SameSite=Lax, and a 12-hour session timeout. HttpOnly blocks normal JavaScript reads of the cookie. SameSite=Lax is compatible with the top-level GET return from Google. The local configuration does not explicitly enable a Secure cookie; a deployed HTTPS setup should configure that.

CORS accepts the configured frontend origin plus the listed localhost/127.0.0.1 origins and permits credentials. CORS controls browser cross-origin access; it does not replace authentication or CSRF. The normal Vite proxy avoids cross-origin frontend API calls.

> **Encryption's practical boundary**
>
> The running backend can decrypt the refresh token. If both the database and encryption key are compromised, the token can be recovered. The current cipher does not bind the encrypted value to account_id with additional authenticated data.

The endpoints at a glance

All paths below are backend paths. The frontend prepends /api through its shared API helper.

Method + path	Input / result	Protection
GET /api/auth/csrf	Returns token and headerName; prepares a session token.	Public route.
GET /api/gmail/status	Returns {connected, email}. Reads the local row.	Session login.
POST /api/gmail/connect	JSON {email}; returns {url} for Google navigation.	Login + CSRF + saved profile.
GET /api/gmail/callback	Google query parameters code/state or error; sends a browser redirect.	Public filter route; controller checks session, state, owner, email.
DELETE /api/gmail/connection	Deletes local connection and cached emails; returns 204.	Login + CSRF.
POST /api/emails/fetch	Optional query and maxResults URL parameters; returns fetched and stackSize.	Login + CSRF + saved connection.
GET /api/emails/stack	Returns the current account's cached EmailDto array.	Session login.
GET /api/emails/stack/latest	Returns the first cached item, or 204 if empty.	Session login.

Example responses illustrative example

```
GET /api/gmail/status
{"connected":true,"email":"career.mailbox@gmail.com"}

POST /api/emails/fetch
{"fetched":2,"stackSize":2}

GET /api/emails/stack
[{"messageId":"example-id","subject":"Engineer opening",
  "from":"Hiring <jobs@example.com>","snippet":"A new role...",
  "body":"Full message text...","receivedAt":"2026-10-03T10:30:00"}]
```

fetched is the number returned by this fetch operation. Repeated fetches can return the same IDs, so this count is not necessarily the number of newly discovered messages. stackSize is the resulting cached collection size.

Configuration you can trace

These are placeholder names and local URLs; the guide contains no real credential values.

Environment variable	Purpose
GOOGLE_CLIENT_ID	The web application OAuth client ID.
GOOGLE_CLIENT_SECRET	Its matching client secret, used only by the backend.
GMAIL_TOKEN_ENCRYPTION_KEY	Base64-encoded 32 random bytes; must remain stable to decrypt existing tokens.
GOOGLE_REDIRECT_URI	Defaults to http://localhost:8080/api/gmail/callback.
APP_FRONTEND_URL	Defaults to http://localhost:5173; used when redirecting back to the UI.

application.properties imports the ignored jobgraph-ai-service/.env as a properties file using two optional paths, supporting startup from the repo root or backend folder. Spring maps the variables above into app.gmail.* and injects them through @Value. Editing .env requires restarting the backend to reload these values.

Local Google configuration

1. Enable Gmail API in the Google Cloud project used by the OAuth client.
2. Use a Web application OAuth client with the callback above registered exactly, including scheme, port, and path.
3. Configure the consent screen and add the Gmail account as a test user if the app is in Testing.
4. Start the frontend at localhost:5173 and backend at localhost:8080; sign in, save a profile, then connect.

Use localhost consistently during the trip. A session established for 127.0.0.1 is associated with a different host from localhost, so a callback to localhost may not recover that session. CORS permission alone does not share cookies between these hostnames.

> **Do not rotate the encryption key casually**
>
> The same saved ciphertext requires the original key. The current format has no key version or migration mechanism. Preserve the key across restarts; plan a token re-encryption or reconnection strategy before replacing it.

Repo: application.properties:23–37; GmailAuthService:37–45; TokenCipher:21–29. Setup [G1]

Diagnose connection failures

First identify which leg failed: your API, Google's screen, or the callback.

Symptom	Where it comes from / what to inspect
401 Login required	Protected API has no valid JobGraph session. Check /api/auth/me and whether the request carries the cookie.
403 before controller breakpoint	Often missing/stale CSRF on a write or a rejected cross-origin request. Inspect the CSRF header and browser Origin/preflight.
409 Save a profile first	POST /gmail/connect rejected an account with no saved profile.
503 during connect	Missing OAuth client configuration or an invalid encryption key can throw IllegalStateException here.
Google redirect_uri_mismatch	Compare the actual authorization URL's redirect_uri with the web client's registered callback exactly.
Google denies a test user	Check consent-screen audience, test-user address, chosen Google account, and organization policy.
gmail=session-expired	Callback cannot recover the required session/login. Check host consistency, session expiry, and whether logout happened.
gmail=invalid-state	Returned state/owner/email session data failed validation; an old callback or second tab can cause this.
gmail=cancelled	Callback had an error or no code. The UI groups these cases into a cancellation notice.
gmail=email-mismatch	Google's confirmed address differs from the typed address.
gmail=error	Token exchange, mailbox lookup, encryption, or persistence failed. The detailed cause is in backend logs.

The UI intentionally exposes short notices. In particular, missing refresh-token failures become the generic gmail=error result even though the service exception explains offline access. Follow the backend exception cause to find the actual reason.

> **A failed replacement**
>
> Completion checks the new address and refresh token before updating the connection. A cancellation or mismatch normally leaves the previous local connection intact.

Repo: GmailConnectionController:73–118; GmailAuthService:73–97. Redirect matching [G1]

Diagnose fetch failures

A saved connection can still fail when the backend tries to use its credential.

Symptom	Meaning and next check
409 Connect Gmail first	No local connection row exists for the authenticated account.
502 Could not fetch Gmail messages	The catch block wraps any fetch exception. Inspect the server cause: refresh failure, API response, network issue, parsing failure, or wrong encryption key.
Google invalid_grant	A refresh token or code is unusable. During fetch, investigate revoked/expired access; reconnect after fixing the cause.
Google quota / permission response	Check the original Google status and reason. The frontend 502 does not distinguish quota, API disabled, or insufficient permission.
Successful fetch, zero messages	Check the actual query, date window, chosen Gmail account, and result limit. No matching messages is a normal empty result.
Messages disappear after restart	EmailStack is memory-only. The saved Gmail connection can remain while cached messages are lost.
HTML-looking body or blank body	Inspect MIME parts. The parser may fall back to HTML text or lack attachment-backed body data.
Incorrect displayed time	The backend converts to LocalDateTime and loses the timezone offset; the browser interprets that local-looking timestamp.

Why reconnect may be needed during development

Google documents that external apps in Testing can receive refresh tokens expiring after seven days when Gmail scopes are requested. Tokens can also stop working after revocation or other account/policy changes. A saved row does not prevent these provider-side changes. [G6]

Useful diagnostics without exposing credentials

- Browser DevTools → Network: inspect method, path, status, response body, and whether the write included its CSRF header.
- Backend log: locate ‘Gmail fetch failed for account …’ or ‘Gmail OAuth callback failed …’, then inspect the nested exception.
- When sharing logs, omit cookies, authorization headers, token response bodies, secrets, and callback codes.

What this version does not promise

These observations come from the implementation, so you can plan the next steps accurately.

Current behavior	Consequence / a possible next step
Manual fetch only; no scheduler or push subscription	New mail appears only after a user fetches. Add a scheduled or event-driven sync if needed.
One list page and sequential detail calls	Large result sets are incomplete and may be slow. Add pagination and controlled batching/concurrency.
In-memory deque per app process	No durable inbox or multi-instance consistency. Persist messages with account and Gmail-connection ownership.
Only each incoming batch is sorted	Later custom queries can disturb global newest-first order. Sort the merged collection or query a durable ordered store.
Disconnect and fetch are separate operations	An in-flight fetch can repopulate the cache after clear/replacement. Bind writes to a connection version and reject stale results.
Status checks the database only	A revoked grant can still look connected. Track reconnect-needed status after provider failures.
Simple MIME and local timestamps	Attachment-backed bodies, encoding, and timezone cases need fuller handling.
One OAuth state slot; no dedicated TTL	Parallel connect tabs can invalidate each other. Store expiring attempts if multi-tab support is needed.
No Google revocation on disconnect	The Google grant can remain. Add explicit revocation if the intended disconnect semantics require it.
No profile-ranking call in this path	A fetched email is not a scored job match. Build the ranking stage as a separate, explicit step.

The code also does not add account ID as AES-GCM additional authenticated data, version encryption keys, configure a provider retry/backoff policy, or set an application-level fetch rate limit. These are design improvements, not claims that this guide implemented them.

> **Scope is still a meaningful limit**
>
> The current Google permission is read-only, but the backend has access to message contents. Treat those contents as user data throughout future filtering, storage, and AI processing.

A debugger tour in your IDE

Use one account and a small test mailbox so each step is easy to follow.

Breakpoint	Inspect and explain
GmailConnectionController.connect()	authentication.getName(), account ID, saved-profile check, and the presence of session attributes after creation.
GmailAuthService.authorizationUrl()	Configured redirect URI, requested scope, login_hint, prompt, and state presence. Avoid copying actual secrets.
GmailConnectionController.callback()	Compare account IDs and state-match result. Watch attributes get removed before token exchange.
GmailAuthService.complete()	Presence of access/refresh tokens, requested vs confirmed address, and whether fallback is taken. Do not log token values.
TokenCipher.encrypt()/decrypt()	Key length=32 bytes, IV length=12 bytes, success/failure of authenticated decryption.
EmailController.fetchEmails()	Current account ID, query string, requested/clamped limit, and returned batch size.
GmailFetchService.fetchEmails()	List-result count, each message ID, payload MIME types, then the oldest-first sorted batch.
EmailStack.pushAll()	Deque contents for only this account and removal of repeated IDs.
InboxPage.showFetchedEmails()	POST fetch succeeds, GET stack returns, React state changes, and the list renders.

Trace failures from the outside inward

1. Find the failing browser request and its HTTP status.
2. Confirm it reached the expected controller breakpoint. If not, inspect authentication, CSRF, CORS, or routing.
3. If it reached a Google .execute(), inspect the provider exception and reason.
4. If Google succeeded, inspect transformation, persistence/cache, and the next frontend request separately.

> **Do not confuse the two localhost services**
>
> The browser usually sends /api requests to Vite at port 5173, which forwards them to Spring at port 8080. Google's registered callback goes directly to Spring at port 8080.

A complete example to rehearse

Fictional account 42 logs in with one email and connects a different Gmail mailbox.

1. **Login:** dev@example.com authenticates to JobGraph. Spring stores its security context in the session. The account's database ID is 42.
2. **Save a profile:** the resume contact email may be contact@example.com. Neither that field nor the Gmail address replaces account ID 42 as the owner.
3. **Connect:** enter career.mailbox@gmail.com. The server saves state S, account ID 42, and that requested address in the session.
4. **Consent:** choose career.mailbox@gmail.com at Google. Google returns code C and state S through the browser to the callback.
5. **Validate:** the session still belongs to account 42 and state matches. Remove the saved attempt; exchange C for tokens. Gmail's profile endpoint confirms career.mailbox@gmail.com.
6. **Persist:** encrypt refresh token R. Save account_id=42, the confirmed address, ciphertext, and updated_at. Redirect to /inbox?gmail=connected.
7. **Fetch:** account 42 clicks Fetch job alerts. Decrypt R, let the Java client refresh an access token, search Gmail, and retrieve matching message details.
8. **Store and show:** convert messages to DTOs, push them into stack[42], return counts, and let React GET /emails/stack to render them.
9. **Different user:** account 99's stack and repository lookup use 99. Supplying another person's resume contact email does not change those lookups.
10. **Disconnect:** delete account 42's local Gmail row and clear stack[42]. The JobGraph account and saved profile remain.

Predict the result before running it

Change the scenario	Expected outcome
Choose a different Google address	email-mismatch; no successful replacement.
Log out before callback	Callback cannot complete for the original session.
Restart after saving the Gmail row	Connection row remains; cached emails need another fetch.
Fetch the same message IDs again	Existing cached copies are removed and refreshed, not duplicated.

Learn with the existing tests

Automated tests explain intent, but mocked Google calls cannot prove live consent works.

Existing test area	What it checks
Authorization URL	Callback, state, offline access, consent parameter, login hint, and Gmail read-only scope.
Callback binding	Wrong state is rejected; a valid callback can use Gmail different from login; cancellation and address mismatch redirect correctly.
Stack and cipher	Accounts get separate stacks; repeated entries are deduplicated; tokens round-trip through encryption; malformed ciphertext is rejected.
AccountProfileIntegrationTests	Session authentication, CSRF behavior, account isolation, session renewal, and allowed frontend preflight handling.

Run the existing suite illustrative example

```
cd jobgraph-backend
mvn test
```

This guide was built by reading source and checking the generated document. It does not claim a new live Gmail consent test or a new backend test run. GmailConnectionTests mocks GmailAuthService.complete() for callback cases, so token exchange, Google address lookup, and refresh-token fallback still need dedicated integration coverage.

Exercises with answers

Question	Answer
Does map(GmailConnection::getGmailAddress) update the row?	No. It converts Optional<GmailConnection> to Optional<String>. An absent connection stays empty.
Why is connect POST, but callback GET?	Connect creates server-side attempt state; Google returns through browser navigation to a validated callback.
Can we decrypt the password like the token?	No. Passwords are hashed for comparison; refresh tokens are encrypted because they must be recovered.

Good next test cases

Cover refresh-token omission, revoked grants, corrupt encrypted tokens, nested MIME, timezones, paging, and fetch-versus-disconnect races. These cases check behavior that the current small test suite does not establish.

Source map and further reading

Use the file map to return from the explanation to the implementation.

Folder	Read these files
backend …/gmail/	GmailConnectionController.java • GmailAuthService.java • TokenCipher.java • GmailFetchService.java • EmailController.java • EmailStack.java • EmailDto.java
backend …/model/ and …/repository/	GmailConnection.java • GmailConnectionRepository.java
backend …/config/ and …/controller/	SecurityConfig.java • CorsConfig.java • AuthController.java; also service/AccountService.java
frontend src/components/	GmailConnectDialog.jsx • ConnectGmailButton.jsx • InboxPage.jsx • FetchedEmails.jsx
Other entry points	frontend src/api/api.js and vite.config.js; backend src/main/resources/application.properties and db/migration/V2__accounts_and_gmail.sql

Backend Java folder prefix: `jobgraph-backend/src/main/java/com/JobGraph/jobgraph_backend/`. Frontend folder prefix: `jobgraph-frontend/`. Code line numbers reflect the snapshot captured on 03 October 2026.

Official references checked for this guide

G1 Google: OAuth for web server applications

G2 Google: Gmail permission scopes

G3 Google: messages.list

G4 Google: messages.get

G5 Google: Message resource and MIME structure

G6 Google: OAuth and refresh-token expiration

S1 Spring Security: CSRF protection

J1 Google Java OAuth client: Credential implementation

Library behavior was also checked in locally installed source JARs: google-api-client 2.7.0 (GoogleOAuthConstants) and google-oauth-client 1.36.0 (Credential). These version-specific sources help distinguish what this application actually uses from examples in newer documentation.

> **Rebuild and keep learning**
>
> The companion HTML is searchable and printable. build_guide.py rebuilds its source excerpts; source-manifest.json records the reviewed file paths and SHA-256 hashes. Re-check explanations and line ranges when application code changes.